Advanced Ensemble Learning

Source: GCI September 2025

1. Bagging (Bootstrap Aggregating)

Bagging (short for bootstrap aggregating) involves repeatedly creating new training datasets by randomly sampling n rows with replacement from the original training dataset of n rows (this process is called **bootstrapping**).

Below is an example of implementing bagging using the breast cancer dataset and constructing a model with the k-NN algorithm. The `BaggingClassifier` class from the `sklearn.ensemble` module is used.

```
from sklearn.datasets import load_breast_cancer
from sklearn.model_selection import train_test_split
from sklearn.ensemble import BaggingClassifier
from sklearn.neighbors import KNeighborsClassifier
import pandas as pd
```

```
# Load the breast cancer dataset
cancer = load_breast_cancer()
X_cancer = cancer.data
y_cancer = cancer.target

print("Dataset loaded successfully!")
print(f"Features shape: {X_cancer.shape}")
print(f"Target shape: {y_cancer.shape}")
```

```
Dataset loaded successfully!
Features shape: (569, 30)
Target shape: (569,)
```

```
# Split into training and test datasets
X_train, X_test, y_train, y_test = train_test_split(
    X_cancer, y_cancer, stratify=y_cancer, random_state=66)

# Setting up the k-NN model and its bagging configuration
models = {
    'kNN': KNeighborsClassifier(),
    'kNN + bagging': BaggingClassifier(KNeighborsClassifier(), n_estimators=100, random_state=0)
}
# Model construction
```

```
scores = pd.DataFrame(columns=["train_score", "test_score"], index=models.keys())
for i, (model_name, model) in enumerate(models.items()):
    model.fit(X_train, y_train)
    scores["train_score"][i] = model.score(X_train, y_train)
    scores["test_score"][i] = model.score(X_test, y_test)

scores
```

```
/tmp/ipykernel_3484/2996474660.py:14: FutureWarning: ChainedAssignmentError: behaviour will change in pandas 3.0!
You are setting values through chained assignment. Currently this works in certain cases, but when using Copy-on-Write (which will become the defau
A typical example is when you are setting values in a column of a DataFrame, like:
df["col"][row_indexer] = value
Use `df.loc[row_indexer, "col"] = values` instead, to perform the assignment in a single step and ensure this keeps updating the original `df`.
See the caveats in the documentation: https://pandas.pydata.org/pandas-docs/stable/user_guide/indexing.html#returning-a-view-versus-a-copy
  scores["train_score"][i] = model.score(X_train, y_train)
/tmp/ipykernel_3484/2996474660.py:14: FutureWarning: Series.__setitem__ treating keys as positions is deprecated. In a future version, integer keys
  scores["train_score"][i] = model.score(X_train, y_train)
/tmp/ipykernel_3484/2996474660.py:15: FutureWarning: ChainedAssignmentError: behaviour will change in pandas 3.0!
You are setting values through chained assignment. Currently this works in certain cases, but when using Copy-on-Write (which will become the defau
A typical example is when you are setting values in a column of a DataFrame, like:

df["col"][row_indexer] = value

Use `df.loc[row_indexer, "col"] = values` instead, to perform the assignment in a single step and ensure this keeps updating the original `df`.
See the caveats in the documentation: https://pandas.pydata.org/pandas-docs/stable/user_guide/indexing.html#returning-a-view-versus-a-copy
  scores["test_score"][i] = model.score(X_test, y_test)
/tmp/ipykernel_3484/2996474660.py:15: FutureWarning: Series.__setitem__ treating keys as positions is deprecated. In a future version, integer keys
  scores["test_score"][i] = model.score(X_test, y_test)
/tmp/ipykernel_3484/2996474660.py:14: FutureWarning: ChainedAssignmentError: behaviour will change in pandas 3.0!
You are setting values through chained assignment. Currently this works in certain cases, but when using Copy-on-Write (which will become the defau
A typical example is when you are setting values in a column of a DataFrame, like:
df["col"][row_indexer] = value
Use `df.loc[row_indexer, "col"] = values` instead, to perform the assignment in a single step and ensure this keeps updating the original `df`.
See the caveats in the documentation: https://pandas.pydata.org/pandas-docs/stable/user_guide/indexing.html#returning-a-view-versus-a-copy
```

In the example above, the `n_estimators` parameter is set to 100, meaning that 100 k-NN models are used for bagging. The training score (`train_score`) remains roughly the same, while the test score (`test_score`) shows an improvement.

The `BaggingClassifier` class also includes parameters such as `max_samples` (default: 1.0) and `max_features` (default: 1.0). The former specifies the proportion of the original dataset to extract during bootstrapping. For example, setting it to 0.5 means 50 samples are drawn from a dataset of 100. The latter specifies the proportion of features sampled, with 0.5 indicating that half of all features are used for training each model.

When the original model is prone to overfitting, introducing diversity among features (for each sample) by not using all the data can serve as an effective countermeasure. Keep this point in mind.

2. Boosting

In bagging, multiple models are independently trained on different bootstrap samples. In contrast, **boosting** generates both training data and models sequentially.

Example of Boosting

The following example demonstrates boosting with a decision tree model (DecisionTreeRegressor). The `AdaBoostRegressor` class from the `sklearn.ensemble` module is used.

Other boosting algorithms include LPBoost, BrownBoost, and LogitBoost, which can be explored further using keywords such as "boosting."

```
from sklearn.tree import DecisionTreeRegressor
from sklearn.ensemble import AdaBoostRegressor
from sklearn.ensemble import StackingClassifier
from sklearn.datasets import fetch_california_housing
```

```
# Load the California Housing dataset
housing = fetch_california_housing()
X_housing = housing.data
y_housing = housing.target

print("Housing dataset loaded successfully!")
print(f"Features shape: {X_housing.shape}")
print(f"Target shape: {y_housing.shape}")
```

```
Housing dataset loaded successfully!
Features shape: (20640, 8)
```

	train_score	test_score
kNN	0.948357	0.923077

```
Target shape: (20640,)
```

kNN + bagging	0.950704	0.937063

```
# Split into training and test datasets
X_train, X_test, y_train, y_test = train_test_split(
    X_housing, y_housing, random_state=66)

# Configuring parameters for a decision tree and AdaBoostRegressor
models = {
    'Decision tree': DecisionTreeRegressor(random_state=0),
    # Specify the base model in the `estimator` argument (a decision tree is used here, but other models are also applicable)
    'AdaBoost': AdaBoostRegressor(estimator=DecisionTreeRegressor(), random_state=0)
}

# Model configuration
scores = {}
for model_name, model in models.items():
    model.fit(X_train, y_train)
    scores[(model_name, 'train_score')] = model.score(X_train, y_train)
    scores[(model_name, 'test_score')] = model.score(X_test, y_test)

# Display results (R2 score)
pd.Series(scores).unstack()
```

	test_score	train_score
AdaBoost	0.821205	0.999373
Decision tree	0.613043	1.000000

When using a standalone decision tree, the test score is approximately 0.61. Switching to ensemble learning with the AdaBoostRegressor class improves the test score to around 0.82, highlighting the effectiveness of ensemble learning in accuracy-critical scenarios.

However, note that the AdaBoost result shows a slight tendency toward overfitting (a discrepancy between training and test scores). This is a point to consider.

3. Stacking

Unlike Bagging and Boosting, Stacking combines different types of models using another model (called a meta-model) to learn how to combine their predictions.

```
# Import libraries
from sklearn.ensemble import (
    StackingRegressor,
    RandomForestRegressor,
    GradientBoostingRegressor
```

```
)
from sklearn.tree import DecisionTreeRegressor
from sklearn.linear_model import RidgeCV
from sklearn.model_selection import train_test_split
from sklearn.datasets import fetch_california_housing
import pandas as pd
```

```
# Load California Housing dataset
housing = fetch_california_housing()
X_housing = housing.data
y_housing = housing.target

# Split dataset
X_train, X_test, y_train, y_test = train_test_split(
    X_housing, y_housing, random_state=66
)

# Define models
models = {
    'Decision Tree': DecisionTreeRegressor(random_state=0),
    'Random Forest': RandomForestRegressor(random_state=0),
    'Gradient Boosting': GradientBoostingRegressor(random_state=0),
    'Stacking': StackingRegressor(
        estimators=[
            ('dt', DecisionTreeRegressor()),
            ('rf', RandomForestRegressor()),
            ('gb', GradientBoostingRegressor())
        ],
        final_estimator=RidgeCV()
    )
}
```

Rather than simply averaging the predictions, RidgeCV learns coefficients for each base model. It also applies regularization to reduce overfitting.

The goal is to see whether stacking can outperform the individual models on unseen data.

Compare the test R^2 scores. Stacking does not guarantee better performance, but it is a good way to understand how multiple different algorithms can work together to improve predictions.

```
# Train and evaluate (this could take a while btw)
scores = {}

for name, model in models.items():
    model.fit(X_train, y_train)
    scores[name] = {
```

```
        'Train R²': model.score(X_train, y_train),
        'Test R²': model.score(X_test, y_test)
    }
```